

Make Sink Function Like Wave Link 3

An implementation prompt for redesigning calmasacow/sink around Elgato Wave Link 3.0 routing semantics.

Repository: <https://github.com/calmasacow/sink> · Target: Elgato Wave Link 3 (desktop mixer), not the discord.py Lavalink library.

How to use this document. Paste the sections below into a coding agent as the system/user prompt. Do not clone Elgato branding, assets, or a pixel-perfect UI. Match the audio model, routing semantics, mix destinations, and operator workflow. Keep Tauri + PipeWire + WirePlumber, GPL-3.0, and ~/.config/sink JSON unless a migration is required.

Goal

Make sink *function* like Elgato Wave Link 3.0 on Linux. Sink is already a PipeWire desktop mixer described as “SteelSeries Sonar for Linux.” Wave Link 3 is a routing-matrix mixer: inputs on the left, independent destination mixes across the top, and monitoring means choosing *which mix you hear*. That matrix — not more EQ — is the distinctive behavior.

Current Sink Model

Today sink is a Sonar-style graph: apps feed named channels; each channel plays to an output device; mixes are extra capture buses for OBS.

```
apps  -->  channels  -->  your ears
                        \-->  a Mix  -->  OBS / recorder
```

Current audio graph.

- Running apps appear automatically and can be assigned to a named channel (Game, Chat, Music, ...). Assignments persist.
- A channel has volume, mute, meters, parametric EQ (up to 10 bands), and **its own output device**. Listening is “this channel plays to this sink.”
- Mixes are extra capture buses for OBS. Master Mix = everything. Custom mixes can exclude channels (“everything except music”) and stay current as channels change.
- A mix can include the processed microphone (Sonar-style Stream Mix).
- Each mix member has independent send level + mute, so recorders hear a different balance than you do.
- A mix can also sit among output devices and stay capturable via its monitor.
- Microphone path: noise gate → compressor → limiter → virtual mic selectable in Discord/OBS.
- Profiles save/switch full layouts from the tray. Themes exist (Original, Tokyo Night, Gruvbox Dark).

Target Wave Link 3 Model

Wave Link 3 is a **routing matrix**, not a “channel → one output” mixer. Inputs and mixes are separate first-class objects. Each mix is both a virtual capture device and a bus that can play to one or more physical outputs.

INPUTS (left)	MIXES (top)	OUTPUTS
-----	-----	-----
Hardware mics / interfaces	Personal / Monitor mix	Headphones
Capture cards	Stream mix	Speakers
Software channels	Chat / Discord mix	Capture card
(unlimited apps grouped	VOD / clean rec mix	Virtual capture
into one channel)	MicFX / custom mixes	devices per mix

Target routing matrix.

Rule 1 — Inputs and mixes are separate objects

- Inputs = sources (mic, interface, capture card, software channel).
- A software channel groups unlimited apps under one fader.
- Mixes = destination buses. Default starter set: Personal, Stream, and Chat. User can add, rename, and icon custom mixes.
- Practical cap can stay generous (Wave Link: 8 software input channels + hardware inputs; 5 mixes). Do not hard-limit to 2 mixes.

Rule 2 — Routing is a matrix, not a single output picker

- Every input × mix cell has: routed on/off, send fader, send mute.
- Changing Stream levels must never change Personal levels.
- Muting an input on Stream must not mute it in Personal.
- The input's own strip fader is the **source** level that feeds every mix (pre-send), matching Wave Link's left-side channel fader.

Rule 3 — Monitoring is “which mix am I listening to?”

- One (or more) monitor outputs attach to the **currently monitored mix** (ear icon / listen toggle on each mix).
- Switching monitor from Personal → Stream must instantly change what the headphones hear, without rewriting app → channel routing.
- Each mix can also be sent to one or more physical outputs at the same time (headphones + speakers + capture-card line).
- Each mix always exists as a virtual capture source for OBS / Discord / Zoom. OBS adds the Stream mix as a **mic input**, not Desktop Audio.

Rule 4 — Mixes are independent graphs

- **Personal** = what you want in your ears (often Discord loud, music present, mic optional/low).
- **Stream** = what the audience hears (mic forward, music ducked or lower, no notification spam unless desired).
- **Chat** = what friends hear on Discord (usually processed mic ± soundboard; usually NOT game/music/system).
- Extra mixes for VOD (no copyrighted music) or MicFX are first-class, not special-cased one-offs.

Rule 5 — Effects live on the input

Processed audio then travels into every mix that input feeds.

- Keep sink's per-channel parametric EQ.

- Keep mic gate / compressor / limiter, but attach them to the mic **input**, not only to a single virtual-mic node that bypasses the matrix.
- After processing, the mic is just another input that can be routed independently into Personal / Stream / Chat.
- Linux substitute for Wave Link’s VST3/AU rack: a per-input FX chain of built-in modules first (EQ, gate, comp, limiter, high-pass, optional de-esser). Leave a clean hook for future LV2/LADSPA. Do not block the redesign on a full VST host.

Rule 6 — Hardware and control-surface scope

Hardware Wave-device pages, Stream Deck marketplace, Voice Focus cloud AI, Siri, and Elgato firmware are out of scope unless already present. Replace with Linux equivalents only where they affect *function*:

- Any USB/PipeWire mic or interface is a hardware input.
- Capture-card audio is a hardware input.
- “Hide unused devices” so HDMI monitors / webcams stop cluttering the picker.
- Optional later: Stream Deck / CLI / D-Bus / a local JSON-RPC so faders and mutes can be driven externally. Design backend commands as if a control surface will exist.

Required PipeWire Graph Changes

Rebuild the PipeWire object graph around buses. Stop treating “channel output device” as the primary routing primitive. A channel no longer owns a destination sink. Mixes do.

```

App streams  --> Software Input  (channel node + optional EQ/FX)
Mic / HW in  --> Hardware Input (node + FX chain)
                |
                +-- send --> Mix A  (null sink / loopback mix)
                +-- send --> Mix B
                +-- send --> Mix N
                    |
                    +-- virtual source "Sink: <mix name>"
                    |   (what OBS / Discord select)
                    +-- playback links to 0..N physical sinks
                        (monitored mix and any mix marked
                         as a hardware output)

```

New PipeWire object graph.

Graph requirements

- App assignment still works the Sonar/Wave way: pick an app, drop it on a software input; remember it forever by application id / binary / PipeWire media.name with a stable fallback.
- Master Mix, if kept, is just another mix whose default routing is “all inputs on.” It is not a hidden special bus.
- Mix membership must be dynamic: adding a new input does not automatically contaminate Chat or VOD. New inputs default to Personal + Stream ON, Chat OFF, unless the user changes the default.
- Send levels are independent of the input fader. Input fader moves the source; mix-cell faders trim per destination.
- Latency: prefer PipeWire links / combine-stream / filter-chain over extra userspace copies. Monitoring the Personal mix must stay low-latency enough for voice + game.

- Meters: peak + peak-hold on every input, every mix, and the monitor output. Mute state must be visually obvious (Wave Link turns muted rows red).
- Destroying a mix must tear down its virtual source and playback links without breaking remaining mixes.
- Profiles must snapshot: inputs, app assignments, FX, the full routing matrix, mix names/icons, monitor selection, and output bindings.

Required UI Changes

Replace the Sonar-style “channel strip + optional mix pop-out” with Wave Link 3’s two views.

A. Routing table (default)

- Inputs as rows on the left (name, icon, input fader, mute, FX, meter).
- Mixes as columns across the top (name, icon, mix fader, mute, ear/monitor toggle, output assignment).
- Cell at each intersection: add/remove from mix, send fader, send mute.
- “+” to add input or mix.
- Drag-reorder inputs and mixes.

B. Compact mixer

- Essential faders only: inputs + the monitored mix + mix masters.
- For live use when the matrix is too wide.

Also required

- Monitor device picker at the top (physical sinks for the listened mix).
- Multi-output per mix: a mix can play to headphones and a capture card together.
- Per-input FX drawer (EQ curve already exists; add gate/comp/limiter for mic inputs and a high-pass).
- Hide/deactivate unused system devices.
- Rename + custom icon for inputs and mixes.
- Sound Check: record a short clip of the selected input, play it through the monitored mix so FX can be tuned without going live.

Migration

On first launch after the change:

- Existing channels become software inputs.
- Existing “channel output device” becomes the Personal mix’s playback device.
- Existing custom mixes become named mixes; their send levels become matrix cell values.
- Existing processed virtual mic becomes the mic input routed to a Chat mix by default, and also available to Stream/Personal.
- Keep a backup of the previous ~/.config/sink JSON.

Non-Goals

- Windows/macOS Wave Link protocol or Elgato hardware DSP protocol.

- Bit-identical UI.
- Unlimited VST host in v1.
- Changing package name / breaking AUR–COPR install paths unless necessary. Command names and virtual device names may change; document the new device names clearly (“Sink: Stream”, “Sink: Chat”).

Acceptance Tests (Manual)

1. Launch a game, Discord, Spotify, a browser. Assign them to Game, Chat, Music, System inputs. Assignments survive restart.
2. Create Personal, Stream, Chat mixes. Personal: Game 80, Discord 100, Music 40, Mic 20. Stream: Game 70, Discord 0, Music 25, Mic 100. Chat: Mic 100, optional soundboard; Game/Music off.
3. Headphones hear Personal. Switching the ear icon to Stream changes only the headphones. Discord and OBS still see their own mix devices.
4. OBS captures “Sink: Stream” as a mic input and does not need Desktop Audio. Stream balance is independent of headphone balance.
5. Discord input is “Sink: Chat” and does not contain game audio.
6. Muting Music on Stream leaves Music playing in headphones.
7. Adding a new app to System does not automatically appear in Chat.
8. Mic gate/comp/limiter affect every mix the mic is routed to.
9. A mix can be played to two physical devices at once.
10. Profile switch restores the entire matrix.
11. Unused HDMI/webcam devices can be hidden.
12. No regressions in PipeWire graph cleanup on quit (no orphaned null sinks / loopbacks).

Implementation Style

- Read the current src-tauri PipeWire graph code and src UI first. Propose the new data model (Input, Mix, RouteCell, OutputBinding, Profile) before rewriting UI.
- Keep types strict. One source of truth in the Rust backend; the frontend renders state and sends commands.
- Prefer incremental graph migration with feature flags only if it reduces risk; the user-facing model should ship as Wave Link-style, not as a hybrid of Sonar output-per-channel plus a matrix.
- Update README to describe the new model. Include a short “coming from Sonar / old sink” mapping.
- Start by writing the data model and a graph diagram of the new PipeWire objects, then implement backend commands, then the routing table UI.

Notes for the Human Operator

If the intended target was instead **Wavelink 3**, the discord.py Lavalink wrapper, this document is the wrong brief. Sink is a desktop PipeWire mixer; that would be a different project.

If an agent only adds mix sends on top of per-channel output devices, the result will still feel like Sonar. Reject that hybrid. The listen path must be “monitor this mix,” not “each channel picks a sink.”